

חלק 3 - Docker Compose

מאפשר לך לנהל מספר קונטיינרים באמצעות קובץ תצורה יחיד Docker Compose.

במקום להריץ מספר פקודות `docker run` ארוכות, אתה מתאר הכל בקובץ אחד: `docker-compose.yml`

(או, בגרסאות Docker חדשות יותר, מקובל להשתמש ב: `compose.yml`)

לאחר מכן מפעילים הכל באמצעות הפקודה:

```
docker compose up
```

למה להשתמש ב-Docker Compose?

תאר לעצמך אפליקציית Node.js שזקוקה ל:

- Node.js
- MongoDB
- Redis

ללא Compose: (שלוש פקודות נפרדות)

```
docker run ...  
docker run ...  
docker run ...
```

עם Compose: פקודה אחת בלבד.

```
docker compose up
```

הכל מתחיל לעבוד יחד.

מבנה בסיסי (Basic Structure)

```
services:
  app:
    image: node:22
  db:
    image: mongo
```

הכל יושב תחת מקטע ה- `services`.

פרויקט לדוגמה (Example Project):

```
project/
├── Dockerfile
├── docker-compose.yml
├── server.js
└── package.json
```

בנייה מתוך Dockerfile

במקום להשתמש בתמונה קיימת:

```
services:
  app:
    build: .
```

יבצע Compose `docker build` באופן אוטומטי.

Ports (פורטים)

```
ports:
  - "3000:3000"
```

שווה ערך להרצה: `docker run -p 3000:3000`

Environment Variables (משתני סביבה)

```
environment:  
  PORT: 3000  
  NODE_ENV: production
```

שווה ערך להרצה: `docker run -e PORT=3000`

Container Name (שם הקונטיינר)

```
container_name: my-api
```

במקום שם שנוצר באקראי על ידי המערכת.

depends_on (תלויות / סדר הפעלה)

שולט בסדר ההפעלה של השירותים.

```
services:  
  app:  
    depends_on:  
      - db  
  db:  
    image: mongo
```

יפעיל את `db` לפני שיפעיל את `app`.

הערה חשובה: `depends_on` שולט רק בסדר ההפעלה. הוא **לא** ממתין עד שמסד הנתונים יהיה מוכן לחלוטין לקבל חיבורים.

Networks (רשתות)

קונטיינרים באותו פרויקט Compose מחוברים לרשת ברירת מחדל באופן אוטומטי, כך שהם יכולים לתקשר באמצעות שמות השירותים שלהם.

```
services:
  app:
    image: node
  db:
    image: mongo
```

בתוך קונטיינר ה-`app`, תוכל להתחבר ל-MongoDB באמצעות הכתובת: `mongodb://db` [27017](#)

ניתן גם להגדיר רשתות מותאמות אישית:

```
networks:
  backend:

services:
  app:
    networks:
      - backend
  db:
    networks:
      - backend
```

Volumes (ווליומים)

ווליומים שומרים על הנתונים (Persist Data) גם אם הקונטיינרים נמחקים.

```
services:
  db:
    image: mongo
    volumes:
      - mongo-data:/data/db
volumes:
  mongo-data:
```

הנתונים של MongoDB יישארו זמינים מעבר ליצירה מחדש של הקונטיינר.

דוגמה מלאה (Complete Example)

```
services:
  app:
    build: .
    container_name: node-api
    ports:
      - "3000:3000"
    environment:
      PORT: 3000
    depends_on:
      - db
  db:
    image: mongo
    container_name: mongodb
    volumes:
      - mongo-data:/data/db

volumes:
  mongo-data:
```

פקודות Docker Compose נפוצות

הפעלת שירותים: `docker compose up`

הפעלה ברקע (Background): `docker compose up -d`

עצירת שירותים: `docker compose down`

בניית תמונות: `docker compose build`

הצגת שירותים רצים: `docker compose ps`

הצגת יומנים (Logs): `docker compose logs`

מעקב אחר יומנים בזמן אמת: `docker compose logs -f`

הפעלה מחדש של שירותים: `docker compose restart`

תהליך העבודה הכולל (Overall Workflow)

Single Container

Write Dockerfile

|

[docker build]



Docker Image

|

[docker run]



Container

Multiple Containers

docker-compose.yml

|

[docker compose up]



App + Database + Redis + Other Services